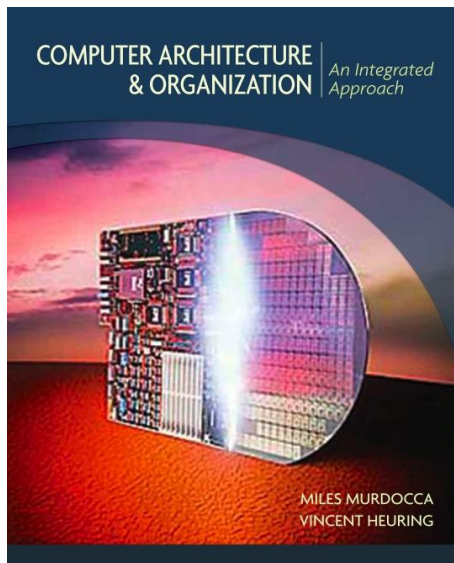


# Organización de las Computadoras

***Carolina Chaves Garcia***

---



## Interrupciones y Excepciones

# *Introducción*

# Introducción

## ¿Qué es una interrupción?

- Una interrupción detiene temporalmente el flujo normal de ejecución del código para atender un **evento externo o interno**, como una señal de un pulsador, temporizador, UART, etc. Al ocurrir, se ejecuta una **rutina de atención a interrupciones** (ISR, *Interrupt Service Routine*).
- Una interrupción es una señal externa que le dice al procesador: “**¡Detente un momento y haz otra cosa urgente!**”
- Las interrupciones permiten que el procesador responda a eventos externos sin necesidad de polling (consultar constantemente).

En ESP32-C3, trabajaremos principalmente con:

- Interrupciones externas (GPIO)
- Interrupciones de temporizador
- Interrupciones de periféricos (UART, etc.)

# Introducción

## *¿Qué es una interrupción?*

- Las interrupciones en RISC-V son eventos que interrumpen el flujo normal de ejecución de un programa para atender situaciones especiales, como
  - Entradas de usuario
  - Temporizadores
  - Errores.
- Ejemplos:
  - Un temporizador terminó su cuenta.
  - Llega un dato por UART
  - Se presiona un botón.
- El procesador suspende su flujo normal, ejecuta una rutina especial llamada ISR (Interrupt Service Routine) y luego retoma su ejecución.

## *¿Qué es una excepción?*

- **Eventos generados** internamente por la **CPU cuando ocurre un error o condición especial**. Ocurren cuando la CPU detecta un problema mientras ejecuta una instrucción.
- A diferencia de las interrupciones (que pueden venir de eventos externos como el teclado o un temporizador), las excepciones son generadas internamente por el procesador.
- **Ejemplos comunes de excepciones**
  - Acceso ilegal a la memoria (leer/escribir en una dirección inválida).
  - Ejecución de una instrucción ilegal (una instrucción no soportada).
  - División por cero.
  - Acceso a una dirección mal alineada (por ejemplo, cargar un entero de 4 bytes en una dirección no múltiplo de 4).
  - División por cero.
  - Instrucción ilegal.
  - Violación de memoria.
  - Llamada al sistema (ecall).

# *Secuencia de activación*

# Secuencia de activación



# *Registros de estado y control del procesador*



## Registros de estado y control del procesador

- Son registros especiales que usa el procesador para gestionar su propio estado interno y responder a eventos como
  - o interrupciones
  - o excepciones
  - o llamadas al sistema
  - o cambios de modo, etc.
- RISC-V los agrupa bajo los CSRs (Control and Status Registers).

| Registro       | Nombre                            | Uso principal                                                                            |
|----------------|-----------------------------------|------------------------------------------------------------------------------------------|
| <b>mstatus</b> | Machine Status                    | Guarda bits de control del estado global (modo actual, interrupciones habilitadas, etc.) |
| <b>mtvec</b>   | Machine Trap-Vector Base Address  | Dirección de salto cuando ocurre una interrupción o excepción                            |
| <b>mepc</b>    | Machine Exception Program Counter | Dirección de la instrucción que causó la excepción                                       |
| <b>mcause</b>  | Machine Cause                     | Código de la causa de la excepción/interrupción                                          |
| mtval          | Machine Trap Value                | Información adicional (como dirección de acceso inválido)                                |
| mie            | Machine Interrupt Enable          | Habilita interrupciones específicas                                                      |
| mip            | Machine Interrupt Pending         | Indica si hay interrupciones pendientes                                                  |

# *Interrupciones en ESP32-C3*

- El ESP32-C3 no usa el esquema de vectores rígidos del modelo estándar de RISC-V (CLINT) sino un sistema más rico:
  - El controlador de interrupciones permite hasta 31 interrupciones asíncronas
  - Cada interrupción puede configurarse con prioridad (niveles 1 a 15) y tipo (nivel o flanco).
    - Interrupción por nivel: La interrupción se mantiene activa mientras el nivel de la señal esté presente. Si la condición (nivel alto o bajo) persiste, la interrupción puede generarse continuamente. # Nivel bajo (0) - Se activa mientras GPIO = 0
    - Interrupciones por Flanco: La interrupción se activa solo en la transición (flanco de subida, bajada o ambos). Una vez detectado el flanco, la interrupción se marca incluso si la señal vuelve a su estado original. # Flanco de subida (0→1)

# Interrupciones en ESP32-C3

## *Registros importantes*

- Hay registros mapeados en memoria para habilitar/deshabilitar, limpiar, estado pendiente, etc.
- Por ejemplo, el registro INTERRUPT\_CORE0\_CPU\_INT\_ENABLE\_REG (nombre genérico) controla el estado de habilitación de interrupciones por núcleo.
- El registro INTERRUPT\_CORE0\_CPU\_INT\_CLEAR\_REG sirve para limpiar las interrupciones de tipo flanco (toggle) cuando han sido atendidas.
- **INTERRUPT\_CORE0\_SOURCE\_X\_MAP\_REG**
- **INTERRUPT\_CORE0\_GPIO\_INTERRUPT\_PRO\_NMI\_MAP\_REG**
- **INTERRUPT\_CORE0\_TG\_T0\_INT\_MAP\_REG**
- **INTERRUPT\_CORE0\_CPU\_INT\_ENABLE\_REG**
- **INTERRUPT\_CORE0\_CPU\_INT\_CLEAR\_REG**
- **INTERRUPT\_CORE0\_CPU\_INT\_TYPE\_REG**
- **CPU\_INT\_ID**

# Interrupciones en ESP32-C3

## *Registros importantes*

Fuente de interrupción  
(GPIO, periférico, etc)  
**detecta evento** (por  
flanco o nivel).

Esa señal es enviada  
al módulo de  
**Interrupt Matrix** del  
ESP32-C3.

En la matriz, la señal de la fuente  
(**Source\_X**) es **mapeada** a un ID  
de interrupción del CPU  
(**Interrupt\_P**).

Dentro del controlador de interrupciones del CPU:

- Se verifica si la interrupción está habilitada.
- Se verifica umbral/mascara/prioridad.
- Se marca la interrupción como pendiente si todo está habilitado (**EIP\_STATUS**)

CPU ve la interrupción  
pendiente, guarda  
contexto y salta al vector  
apropiado.

En **ISR** se debe **limpiar la fuente**  
(**INT\_CLEAR\_REG**) para que esa  
señal no quede pendiente  
eternamente.

Salida de la ISR con instrucción  
**mret** restaurando el contexto y  
continuyendo con el flujo normal.

## *Pasos para configurar una interrupción GPIO (un pulsador)*

- Configurar el pin GPIO como entrada.
- Habilitar la interrupción en el pin (Ejm: flanco de subida).
- Registrar la dirección de la ISR en el vector de interrupciones.
- Habilitar las interrupciones globales.

# Interrupciones en ESP32-C3

## Ejercicio

- Encender un LED en el GPIO1 con un pulsador en el GPIO3.
- **Completa el código (las direcciones las puedes cargar como desees)**

```
.section .text
```

```
.global gpio_irq_init
```

```
.global gpio_isr
```

```
# === DEFINICIONES DE BASES Y REGISTROS ===
```

```
.equ GPIO_BASE,          0x60004000          # Base para registros GPIO
```

```
# Registros para configurar entrada/salida
```

```
.equ GPIO_ENABLE_W1TS_REG, (GPIO_BASE + 0x__) # Escribir 1 para activar salida (Set)
```

```
.equ GPIO_ENABLE_W1TC_REG, (GPIO_BASE + 0x__) # Escribir 1 para desactivar salida (Clear)
```

```
# Registros para salida de LED
```

```
.equ GPIO_OUT_W1TS_REG, (GPIO_BASE + 0x__)    # Escribir 1 para poner HIGH
```

```
.equ GPIO_OUT_W1TC_REG, (GPIO_BASE + 0x__)    # Escribir 1 para poner LOW
```

```
# Registros de interrupción del GPIO
```

```
.equ GPIO_STATUS_REG, (GPIO_BASE + 0x__)      # Estado de interrupciones pendientes
```

```
.equ GPIO_STATUS_W1TC_REG, (GPIO_BASE + 0x__) # Limpieza de interrupción (Write-1-To-Clear)
```

```
.equ GPIO_PCPU_INT_REG, (GPIO_BASE + 0x__)    # Interrupciones de GPIO ya "propagadas" al
```

```
PRO CPU
```

# Interrupciones en ESP32-C3

## *Ejercicio*

# Registros del controlador de interrupciones del CPU / matriz

```
.equ INTR_MAP_BASE,          0x6000F000
.equ INTERRUPT_CORE0_CPU_INT_ENABLE_REG, (INTR_MAP_BASE + 0x0104)
.equ INTERRUPT_CORE0_CPU_INT_CLEAR_REG, (INTR_MAP_BASE + 0x010C)
.equ INTERRUPT_CORE0_CPU_INT_TYPE_REG, (INTR_MAP_BASE + 0x0108)
.equ CPU_INT_ID, 10          # Canal de interrupción del CPU elegido
.equ GPIO_BTN_NUM, 3         # Pin GPIO para el botón
.equ GPIO_LED_NUM, 1         # Pin GPIO para el LED
```

# Máscaras binarias

```
.equ GPIO_BTN_MASK, (1 << GPIO_BTN_NUM) # 0b00001000 → máscara para GPIO3
.equ GPIO_LED_MASK, (1 << GPIO_LED_NUM) # 0b00000010 → máscara para GPIO1
```



# Interrupciones en ESP32-C3

## *Ejercicio*

```
# =====
```

```
# INICIALIZACIÓN: GPIO + IRQ
```

```
# =====
```

```
gpio_irq_init:
```

```
    # Configurar GPIO3 como entrada
```

```
    _____ # Cargar la máscara del botón en t0
    _____ # Cargar parte alta de la dirección GPIO_ENABLE_W1TC
    _____ # t1 ← dirección completa
    _____ # Clear bit correspondiente → GPIO3 como entrada
```

```
    # Configurar GPIO1 como salida (LED)
```

```
    _____ # Cargar la máscara del LED en t0
    _____ # Cargar parte alta de la dirección GPIO_ENABLE_W1TS
    _____ # t1 ← dirección completa
    _____ # Set bit correspondiente → GPIO1 como salida
```

```
    # --- Habilitar canal 10 de interrupción del CPU ---
```

```
    _____ # t1 (bit 10 activado) CPU_INT_ID
    _____ # t0 = parte alta de INT_ENABLE
    _____ # t0 = dirección completa
    _____ # habilita canal 10 del CPU
```

# Interrupciones en ESP32-C3

## *Ejercicio*

# --- Configurar tipo de interrupción (flanco) para canal 10 ---

|       |                                          |
|-------|------------------------------------------|
| _____ | # t0 = parte alta de INT_TYPE            |
| _____ | # t0 = dirección completa                |
| _____ | # tipo: flanco ( 1 = edge-triggered)     |
| _____ | # volver de la función de inicialización |

# =====

# RUTINA DE SERVICIO DE INTERRUPCIÓN (ISR)

# =====

gpio\_isr:

# --- Leer estado de interrupciones GPIO ---

|       |                                                             |
|-------|-------------------------------------------------------------|
| _____ | # t0 = parte alta de GPIO_STATUS_REG                        |
| _____ | # t0 = dirección completa                                   |
| _____ | # t1 = contenido del registro de estado (interrupt pending) |

# --- Verificar si GPIO3 fue quien disparó la interrupción ---

|       |                                                          |
|-------|----------------------------------------------------------|
| _____ | # t2 = t1 & GPIO_BTN_MASK → verificar si se activó GPIO3 |
| _____ | # si t2 == 0, no fue GPIO3 → salir de la ISR             |

# Interrupciones en ESP32-C3

## Ejercicio

# --- (Opcional) Leer GPIO\_PCPU\_INT\_REG para confirmar interrupción activa ---

\_\_\_\_\_ # t3 = parte alta del registro de interrupción CPU  
 \_\_\_\_\_ # t3 = dirección completa  
 \_\_\_\_\_ # t4 = valor actual (qué GPIO interrumpió al CPU)

# --- Encender LED (GPIO2 HIGH) ---

\_\_\_\_\_ # t5 = GPIO\_LED\_MASK → máscara de GPIO1  
 \_\_\_\_\_ # t6 = parte alta de dirección de salida (set)  
 \_\_\_\_\_ # t6 = dirección completa  
 \_\_\_\_\_ # salida en GPIO2 en HIGH → enciende LED

# --- Limpiar interrupción de GPIO3 ---

\_\_\_\_\_ # t7 = GPIO\_BTN\_MASK → máscara de GPIO3  
 \_\_\_\_\_ # t8 = parte alta de STATUS\_W1TC  
 \_\_\_\_\_ # t8 = dirección completa  
 \_\_\_\_\_ # limpia la interrupción del GPIO3 (Write 1 to clear)

# --- Limpiar interrupción a nivel CPU (canal 10) ---

\_\_\_\_\_ # t0 = parte alta INT\_CLEAR  
 \_\_\_\_\_ # t0 = dirección completa  
 \_\_\_\_\_ # t1 = 0x400 → bit 10  
 \_\_\_\_\_ # limpia interrupción del CPU en canal 10

isr\_exit:

mret

# retornar de la interrupción al programa principal mret

# Gracias